# PaperJury: Adversarial Due-Process Pre-Submission Review and Bounded Auto-Revision for LaTeX CS Papers

Yiran Wang

**Abstract**

Forward-only AI writing tools cannot return "no fix": a yes-and rewriter is structurally unable to tell an author that a concern is invalid, and asking an LLM whether a paper is good elicits sycophancy or an unbounded nitpick flood. We present PaperJury, a Claude Code skill that brings adversarial, due-process pre-submission review and bounded auto-revision to LaTeX CS papers. PaperJury is realized as Skill + Workflow + Memory on a general agent harness. Because the workflow sandbox has no filesystem or subprocess, every trustworthy guard (ledger, journal, apply-patch, anchor-diff, cross-reference, spine, compile, decompose, compliance) runs deterministically orchestrator-side in Node between schema-validated semantic fan-out workflows, with the ledger row as the single source of truth at apply time. We reframe LLM-as-reviewer as a precision-recall-COST trilemma: once accurate three-way routing dissolves the precision/recall tradeoff, the binding constraint is attention, so PaperJury governs at generation (three holistic domain reviewers read the whole paper once) and at adjudication (deterministic contestability routing sends only substantive-major issues to an expensive trial). The three-way router agrees with expert labels on 94% of trial charges, confirming that accurate routing dissolves the precision/recall tradeoff in practice. Each routed issue is adjudicated by a two-sided trial decided by a near-deterministic verdict (quorum = $\lceil 0.8 \cdot \text{jury} \rceil$, strict $> 60\%$-of-surviving majority) into valid-fixable, author-required, or invalid-drop, with anchor-bounded edit-safety and a deterministically-converged, clerk-merged unattended outer loop. The result is a paradigm in which the load-bearing guards live in deterministic code rather than in agent discretion, and the terminal verdict space can return "no fix" — the move a forward-only rewriter cannot make and a reviewer can.

## 1 Introduction

Every author of a CS-conference paper meets adversarial feedback at the worst possible moment: after submission, when the verdict is fixed and the only remaining move is a rebuttal. The tools available *before* that moment push in the opposite direction. AI writing assistants are forward-only: they draft, polish, compress, expand, and de-AI an existing manuscript [10], but a cooperative rewriter is structurally unable to return the one answer a reviewer can give, namely *"this concern is not real; no fix is warranted."* A yes-and editor that always produces a changed output is the wrong shape for pre-submission hardening, and the wrong shape is reinforced by well-documented model tendencies: hallucinated content [7], sycophancy toward the user's framing [19], and unreliable self-correction that can degrade reasoning when a model critiques itself without an external signal [6].

A natural reaction is to point a critic at the paper. LLM-as-a-judge methods score outputs at scale [26, 13, 5], multi-agent review systems distribute a paper across communicating agents [3, 8], and large studies show that model feedback overlaps non-trivially with human reviewers, especially pre-submission [11, 12]. Yet a critic alone inherits the judge's biases (position, verbosity, self-preference) [26, 13, 16] and, at whole-paper scale, floods: a naive loop-until-dry per-(unit $\times$ lens) prosecution has no governor and scales with candidate count, emitting a flood of candidate charges

that may never go dry. Such volume is not a tuning nuisance; it is the binding constraint, because every downstream adjudicator pays for candidate issues rather than real ones.

**Idea: due process before a reviewer sees the paper.** PaperJury is a Claude Code *skill* that brings adversarial, due-process pre-submission review and bounded auto-revision to LaTeX CS papers. It splits deterministic orchestrator-side guards from schema-validated semantic fan-out to route every issue through contestability-gated routing, a quorum-decided two-sided trial with a three-way verdict, anchor-bounded edit-safety, and a deterministically-converged unattended outer loop. The terminal verdict space includes `invalid-drop` precisely so the system can say "no fix" — the move a rewriter cannot make and a reviewer can. We make five contributions.

**C1: a precision–recall–COST reframing of LLM-as-reviewer.** For whole-paper review the binding constraint is not the precision/recall tradeoff — dissolved by accurate three-way routing into *valid-fixable* / *author-required* / *invalid-drop* — but COST/attention. We reintroduce a governor at GENERATION ($N$=3 holistic domain reviewers, range 2–4, each reading the whole paper once, in place of a naive per-(unit $\times$ lens) loop-until-dry generator that floods) and at ADJUDICATION (deterministic contestability routing that sends only substantive-major issues to the expensive trial).

**C2: a deterministic-vs-semantic split as isolation-by-construction.** Because the Workflow sandbox has no filesystem or subprocess, all trustworthy guards (ledger, journal, apply-patch, anchor-diff, cross-ref, spine, compile-guard, decompose, compliance-check) run orchestrator-side in Node between schema-validated semantic fan-out Workflows, joined by 14 named enrichment seams as the load-bearing data contract, with the ledger row as the single source of truth at apply time.

**C3: a two-sided due-process trial with a mostly-deterministic verdict.** A five-tier trial pairs a whole-paper DEFENSE (which scans the entire manuscript for an "addressed-elsewhere" catch) with 5 decorrelated local-context jurors that always include a most-hostile and a most-charitable framing, even at tier-5, with on-demand context expansion. The validity decision is deterministic (quorum = $\lceil 0.8 \cdot \text{jurySize} \rceil$ and a strict $>60\%$-of-surviving-votes majority, escalating to a 12-juror tier); only the routing of a decided-valid charge consults an agent, with a recall safety net that revives wrong drops and spot-checks strong-consensus majors.

**C4: risk-proportional, anchor-bounded edit-safety.** An AI applies real edits without claim drift via a frozen up-to-7-anchor spine (credited to PaperSpine), a deterministic LOW/RISKY prefilter (anchor-diff plus cross-ref on changed salient tokens), a four-state meaning-audit for anchors and a two-state edit-audit for risky non-anchor edits, atomic exact-once journaled apply with per-edit revert, and an honest DETECT-OR-DEGRADE compile-guard that reports `compiled:null` rather than a false success.

**C5: a convergent, deterministically-gated unattended outer loop.** Clean rounds never show reviewers the ledger, so an independent re-raise is corroboration and a clean re-review is the did-the-edit-fix-it test; a registrar merges on `passage_id` *and* similarity $\geq 0.8$; the `/goal` completion fact is a ledger query (zero gate-blocking majors plus an anti-fake-completion check) rather than a model judgment; and the loop terminates on the first of clerk convergence, applied-quiescence, or hard limits, with author-required issues accumulating to a single human pass.

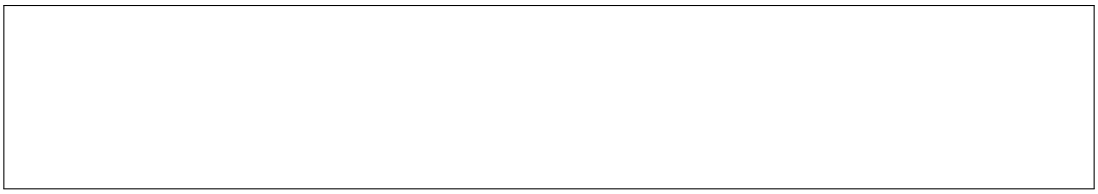

Figure 1: One PaperJury round: deterministic orchestrator-side guards (Node) interleaved with schema-validated semantic Workflows, routing each issue through contestability routing, a two-sided trial with a three-way verdict, recall, and anchor-bounded edit-safety. Placeholder; diagram to be rendered.

**Organization.** The rest of the paper is organized as follows. Section 2 positions PaperJury against AI-assisted writing, LLM-as-judge, automated review, multi-agent debate, and agent harnesses. Section 3 gives the architecture and the deterministic/semantic split. Sections 4–6 detail generation, adjudication, and edit-safety. Section 7 covers the outer loop and auto mode. Section 9 states the methodology by which such a system is evaluated.

## 2 Related Work

PaperJury sits at the intersection of six lines of work. We organize the discussion by theme and, for each, state precisely how PaperJury departs. The recurring distinction is that PaperJury is a *pre-submission self-hardening loop* for a human-authored LaTeX paper, built around an adversarial verdict that can return "no fix," a deterministic state gate, and risk-proportional edit-safety, rather than a forward-only rewriter, a single scoring judge, or an acceptance predictor.

**AI writing assistants.** Human–AI co-writing has been studied as an interaction-design and dataset problem [10], and from-scratch generation now extends to fully autonomous manuscript production inside research agents (below). These treat the model as co-writer or generator. PaperJury is explicitly not a from-scratch drafter: its writing toolkit operates in a constrained direct-edit mode on an existing manuscript (translate, polish, de-AI, compress/expand, caption), always behind explicit author sign-off and with no revision-log or back-translation leakage into the `.tex`. The contribution is review-driven hardening of a human-authored paper, not co-authoring it.

**LLM-as-judge and its biases.** LLM evaluators reach high human agreement at scale on open-ended tasks [26] and on NLG quality via form-filling chain of thought [13], and the design space is now surveyed [5]. The same line documents the failure modes an adjudicator must defend against: position and verbosity bias [26], a bias toward LLM-generated text [13], and self-preference driven by self-recognition [16]. PaperJury does not use a single judge to score a paper. Each contested issue is adjudicated by a two-sided trial—a whole-paper defense versus five decorrelated local-context jurors (escalating to twelve)—with a *deterministic* quorum-and-majority verdict: a charge is decided iff surviving votes $\geq \lceil 0.8 \cdot \mathit{jurySize} \rceil$ and one side holds strictly more than 60% of surviving votes, else it escalates. Reviewers and jurors are isolated from each other but are given the cumulative ledger and prior-round notes for context, which is the structural counter to position and self-preference bias rather than a prompt patch. There is no numeric quality score and no acceptance prediction; the output is a per-issue verdict that can be *invalid-drop* (Contribution C3).

**Automated and AI-assisted peer review.** This is the closest neighborhood. Datasets and probes established the task [9, 12], and a large-scale study showed GPT-4 feedback overlaps non-trivially with human reviewers and is often perceived as helpful, especially pre-submission [11]. Multi-agent review *generation* distributes a paper across communicating agents to write more specific feedback [3], and simulation frameworks model whole review dynamics and reviewer biases [8]. Autonomous research systems embed an automated reviewer as a sub-component [14, 24], and review is used as a training signal in a research–review–refine loop [23, 18]. PaperJury differs on four points. (1) It is pre-submission self-hardening of a human paper, not review generation, review simulation, or an automated reviewer scoring toward an acceptance threshold. (2) Unlike review-as-reward loops [23] and the AI Scientist reviewer [14], it does not predict a score or optimize toward acceptance; review exists to drive adjudicated edits, with completion defined by a deterministic ledger query (zero gate-blocking active majors, plus an anti-fake-completion check that no major remains unadjudicated), not a model judgment (Contribution C5). (3) MARG [3] and AgentReview [8] stop at the review; PaperJury closes the loop with consensus-gated, risk-proportional edits plus an isolated clean-room re-review as the "did the edit actually fix it" test. (4) It reframes the operating constraint as a precision–recall–*cost* trilemma: rather than a fixed per-aspect decomposition that floods, it uses a small number of holistic domain reviewers and deterministic contestability routing so that only substantive-major issues reach the expensive trial (Contribution C1).

**Multi-agent debate and self-refinement.** PaperJury's adjudication machinery descends from this line: iterative self-feedback refinement [15], verbal-reinforcement self-reflection [20], multi-agent debate for factuality and reasoning [4], debate as a better evaluator [2], and principle-guided self-critique-and-revise with RLAIF [1]. These loops are mostly forward-only: critique then revise, biased toward producing a changed output. PaperJury's trial is adversarial and two-sided with a three-way verdict {*invalid-drop*, *valid-fixable*, *author-required*}, so the legitimate output for a charge can be "no edit—this critique is wrong" or "needs new data or an author decision," never a forced rewrite. Debate here adjudicates whether a flaw is real *before* any edit, under isolation and a deterministic vote-counting rule rather than free-form consensus; a separate edit-safety audit then guards the revision. Self-critique drives a defensible verdict, not an automatic revision.

**Agent harnesses and workflows.** PaperJury is realized on a general agent harness. The relevant foundations are interleaved reasoning and acting [25], self-supervised tool use [17], long-horizon agents with skill libraries and self-verification [21], and the broader survey of LLM-based autonomous agents [22]. Rather than a free-form agent, PaperJury splits concerns across three primitives: a Skill (methodology and human gates), Workflows (schema-validated parallel fan-out for reviewers, jury, polish, and recall, with isolation by construction), and Memory (a durable ledger plus learned house-style conventions). Forced by a Workflow sandbox that has no filesystem or subprocess, all load-bearing guards are deterministic Node scripts run orchestrator-side between workflow calls—decompose, ledger gate, anchor-diff, cross-ref, apply-patch, compile-guard—so the agent cannot "reason its way" past the completion gate or the edit-safety check (Contribution C2). This is a deterministic-state machine wrapping stochastic agents, not an autonomous agent improvising tools.

**Hallucination and sycophancy.** These motivate an adversarial verdict over a cooperative rewrite. NLG models hallucinate unintended content [7]; RLHF models are sycophantic, tending to agree with the user or prior framing [19]; and LLMs largely cannot reliably self-correct reasoning without an external signal, sometimes degrading after a self-correction pass [6]. PaperJury treats the

cooperative "yes-and" default as the failure to design against. Three mechanisms operationalize this: the trial can return *invalid-drop*, so a sycophantic agree-and-edit is not the only exit; reviewer and juror isolation blocks the anchoring that drives sycophancy across rounds; and a never-drop recall audit with an independent clean-room re-review counters both the drop-too-much and the hallucinated-issue directions. Because self-correction is unreliable [6], edits are gated by deterministic compile, anchor, and cross-ref checks plus author sign-off rather than by the editing agent's own confidence (Contribution C4). The anchor-bounded design—a frozen up-to-seven-anchor spine with a logic-transfer meaning audit and a minimal-edit revision policy—is inspired by PaperSpine,[1] a motivation-driven forward generate/rewrite tool with no adversarial loop; PaperJury adds the adversarial courtroom and the risk-tiered edit-safety chain on top.

**Positioning.** To our knowledge, PaperJury is distinguished from prior automated-review and self-refine work by being a pre-submission, human-paper-hardening loop built around an adversarial three-way verdict (including "no fix"), a deterministic completion gate and never-drop recall over a durable ledger, risk-proportional edit-safety with author sign-off, and a Skill+Workflow+Memory realization that places the load-bearing guards in deterministic code rather than in agent discretion.

# 3 System Overview

PaperJury is a Claude Code *skill* that brings adversarial, due-process pre-submission review and bounded auto-revision to LaTeX CS papers, splitting deterministic orchestrator-side guards from schema-validated semantic fan-out to route every issue through contestability-gated routing, a quorum-decided two-sided trial with a three-way verdict, anchor-bounded edit-safety, and a deterministically-converged unattended outer loop. This section gives the architecture at a glance; later sections detail each stage.

## 3.1 The closed loop: review → verdict → revise → verify

Forward-only writing assistants are structurally unable to return "no fix": a cooperative rewriter, biased toward producing a changed output, cannot tell the author that a concern is invalid [15, 19]. PaperJury closes the loop instead of pushing the paper forward. *Review* runs $N$=3 holistic domain reviewers (range 2–4) who each read the whole paper once and file evidence-anchored weaknesses. *Verdict* adjudicates each contested issue through a two-sided trial whose output is one of three epistemic routes—`invalid-drop`, `valid-fixable`, or `author-required`—so a judgment call stays with the author and a wrong critique can be dropped rather than forcibly "fixed" [26, 4]. *Revise* applies only `valid-fixable` fixes, under a risk-proportional edit-safety guard and explicit author sign-off. *Verify* re-reviews the edited paper in an isolated clean round: an independent re-raise is corroboration and a clean re-review is the operational "did the edit actually fix it" test, because self-correction without an external signal is unreliable [6]. The loop terminates on a deterministic completion fact, never a model's self-assessment, and on typical CS papers it reaches this fixed point within three rounds.

## 3.2 Three primitives: Skill, Workflow, Memory

The system decomposes into three primitives, each carrying one concern. The **Skill** is the entry point and methodology: the protocol, the reviewer panel, contestability routing, the writing toolkit, and the human gates. **Workflows** are the fan-out engine: every semantic, no-human-in-the-middle

[1] https://github.com/WUBING2023/PaperSpine

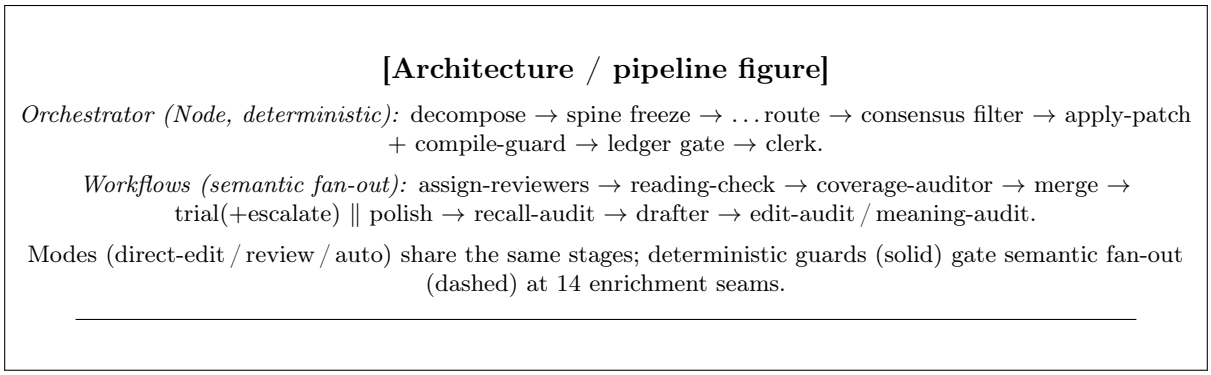


Figure 2: PaperJury architecture. Deterministic Node guards run orchestrator-side between schema-validated semantic Workflows. Every issue flows review → verdict → revise → verify; the ledger row is the single source of truth at apply time, and the deterministic gate (not a model judgment) decides completion.

step (reviewers, trial, polish, recall, merge, clerk) runs as a Workflow, giving parallelism and schema-validated output by construction. **Memory** holds durable state and learned conventions: the *ledger* (a machine `LEDGER.json` source of truth plus a rendered `LEDGER.md` view) and Claude project memory (this paper's house style, venue, and persona tuning). This separation distinguishes PaperJury from free-form agent harnesses that improvise tools [25, 22]: the agent never reasons its way past the completion gate or the edit-safety check.

### 3.3 The deterministic/semantic split and isolation-by-construction

The split is *forced* by the Workflow sandbox: it has no filesystem and no subprocess. All trustworthy guards therefore run orchestrator-side as Node scripts *between* Workflow calls—`ledger`, `journal`, `apply-patch`, `anchor-diff`, `cross-ref`, `spine`, `compile-guard`, `decompose`, and `compliance-check`. Semantic Workflows do agent fan-out only, receiving all inputs inline. The data contract between them is a set of 14 named orchestrator *enrichment seams*, and at apply time the *ledger row is the single source of truth* (the drafter emits neither `passage_id`, `close_criterion`, nor `round`). We exploit the forced split as an *isolation-by-construction* architecture: each agent sees only its own prompt, and every reviewer, juror, and judge prompt additionally carries an explicit isolation instruction to judge only the quoted text and not roam the project. Not-in-prompt is necessary but not sufficient, so both levels are enforced—a structural counter to position and self-preference bias rather than a prompt patch [16, 13].

### 3.4 Three modes of operation

The same machinery is exposed through three modes. **Direct-edit** (the common case) drafts a requested change via the writing toolkit and applies it after author sign-off, with no panel or ledger. **Review** runs one full courtroom round end to end with consensus-gated, author-signed revisions; it does not auto-advance. **Auto** (unattended) runs the same engine under Claude Code's `/goal` across multiple rounds, applying safe fixes under an up-front-authorized bounded-aggressive policy and queuing risky ones, until a deterministic convergence. Auto entry is *explicit only* (it never self-detects), no edit is applied outside the up-front-authorized envelope, and spine anchors are never auto-edited. A cross-mode *submission-readiness* desk-reject screen (`compliance-check.js`, Part A) runs in any mode. Figure 2 shows how the modes share the pipeline.

### 3.5 Why this design

PaperJury reframes the precision/recall tradeoff of LLM-as-reviewer as a precision/recall/*cost* trilemma. Accurate three-way routing dissolves the precision/recall tension; the binding constraint at whole-paper scale becomes cost and attention. We therefore reintroduce a governor at *generation* ($N$ holistic readers, in place of a naive per-(unit $\times$ lens) loop-until-dry generator that has no governor and floods) and at *adjudication* (deterministic contestability routing that sends only substantive-major issues to the expensive trial) [3, 11]. The remaining sections elaborate the five contributions this overview introduces: the trilemma reframing, the deterministic/semantic isolation architecture, the two-sided due-process adjudication engine, the anchor-bounded edit-safety system (whose spine and anchor-audit design is inspired by PaperSpine), and the deterministically-gated unattended outer loop.

## 4 Bounded Generation: Holistic Domain Reviewers

### 4.1 The precision–recall–cost trilemma

LLM-as-reviewer work usually frames quality as a precision/recall tradeoff [11, 12, 5]: nitpick more to raise recall and false positives rise; nitpick less and real flaws slip through. PaperJury dissolves that tension downstream—accurate three-way routing (Section 5) sends every issue to `valid-fixable`, `author-required`, or `invalid-drop`, so a wrong critique is *dropped* rather than scored. With precision and recall decoupled, the binding constraint at whole-paper scale becomes a third axis: **cost/attention**. The natural baseline generator—a per-(unit $\times$ lens) prosecution that loops until it produces no new issue—has no governor: it scales with candidate count and, with no severity floor and no per-unit cap, can flood a whole paper with candidate charges that never go dry. Because every downstream adjudicator pays per *candidate* issue, not per *real* one, an unbounded generator forces a trial sized to thousands of agents. In our runs the bounded $N$-reader generator spawns an order of magnitude fewer agents than the unbounded per-(unit $\times$ lens) baseline while reaching strictly higher issue precision. The fix is a governor at GENERATION.

### 4.2 Bounded $N$ holistic readers

PaperJury replaces the per-(unit $\times$ lens) flood with $N$ domain-expert *holistic* reviewers who each read the *whole paper once*. $N$ defaults to 3 and is hard-clamped to $[2, 4]$ in code, with no per-section reviewer assignment and no per-section coverage quota ($N = \max(2, \min(4, N_{\text{req}}))$). Reviewers are differentiated by professional *subfield* (e.g. "3D vision / neural rendering", "video object segmentation"), not by a generic methodology lens. Each `persona_prompt` is composed in three parts: a project-provided gatekeeper CORE (an "Unflinching Academic Gatekeeper" running a two-pass critique—Pass 1 a blunt fatal-flaw diagnostic, Pass 2 a forensic interrogation of each), a generated per-reviewer DOMAIN OVERLAY (2–4 sentences of the failure modes that subfield is alert to), and the venue style profile. A cross-section mandate ("you MUST reason across sections") is baked into the core, so a holistic reader natively catches abstract-vs-results mismatches, notation clashes, and a contribution the experiments do not validate—there is no separate cross-section agent. `reviewer_id` (R1…RN) is assigned in code, never chosen by an agent, and a default core ships only as a fallback when a project provides none.

**Assignment.** A program-chair agent reads the paper, names $N$ subfields, and writes one overlay per slot. Three guards bound this. A caller-supplied *config-pin* reviewer set wins outright (no

auto-assignment). An optional *verifier* (default on, cap-1 per slot) independently re-derives the subfields and judges each slot `on_topic`; a slot it cannot confirm degrades and its id is recorded in `meta.assignment_unverified`, so one bad slot degrades only itself, never the panel. In interactive review the assignment is surfaced as a *pre-flight* gate before the loop (in-loop prompting is unavailable); headless runs never block, using the degrade path instead. The shipped degrade emits a generic `general CS (gatekeeper)` persona per slot.[2]

### 4.3 Weakness signals and proof-of-reading

Each reviewer returns weaknesses against a fixed schema: `{summary, evidence_anchor, section, significance, kind, references?}`. Two signals drive routing: `significance` (`major` / `minor`) and `kind` (`substantive` / `mechanical`); when unsure between substantive and mechanical the reviewer is instructed to choose substantive, so a genuine concern gets a hearing rather than a copy-edit bypass. The `evidence_anchor` is an exact verbatim quote the weakness rests on: "cannot quote = do not file." Reviewers file no `close_criterion` (the judge sets it later). Exactly *one* `overall_confidence` (integer 1–5) is reported per reviewer, not per weakness; it feeds priority tie-breaking and the recall spot-check.

### 4.4 Anti-skim and anti-drift

The verbatim quote also doubles as a proof-of-reading, enforced in three layers. **L1** is deterministic: every reviewer must return a per-section coverage entry with an `in_section_quote`, and a Node check tags any quote not present in the paper by normalized (whitespace-collapsed, lowercased) substring containment—a cheap string match, never an agent call; a miss triggers a cap-1 re-invoke. **L2** is one *coverage-auditor* that reads all $N$ coverage reports plus the whole paper and flags skimmed `(reviewer, section)` pairs (including cross-reviewer disagreement), judging coverage *quality* only and filing no content charges; every flag is validated against the real `(reviewer, section)` set. **L3** re-invokes reading-check in `targets` mode, cap-1 per flag, with a re-read focus. Orthogonally, *anti-drift* rests on cross-round-stable `passage_id`s from `decompose.js` (a heuristic LaTeX-aware splitter, not a TeX parser), which become the merge and per-passage keys used throughout the pipeline.

### 4.5 Merge: derive in code

After the full anti-skim loop, a *merge* agent clusters the final weakness set. It decides *only* grouping and the clearest framing; every aggregate field is derived deterministically in code. `significance` is the **max** (major dominates) and `kind` is substantive if *any* member is substantive, so a real concern is never diluted by being grouped with milder duplicates. `raised_by_count` (distinct corroborating reviewers) is computed but used *only* for priority order; crucially it **never feeds significance**—more raisers does not promote an issue. `reviewer_confidence` is the max over members and `close_criterion` stays null. Any weakness the agent leaves unclustered (or any invalid index) becomes its own singleton issue, so merge can never silently drop a weakness; the bias is against wrong merges, since a missed merge is recoverable but a wrong merge hides a distinct issue.

[2]The persona documentation describes the degrade fallback as a named Theory/Empirical/Applied triad; the shipped code emits a single generic gatekeeper persona. We follow the code; the triad is doc-level intent.

# 5 Contestability-Routed Adjudication

The trilemma of §3 dissolves the precision/recall tradeoff into accurate routing but makes *cost/attention* the binding constraint. PaperJury attacks cost at adjudication with a single governing principle: spend deep deliberation only where it can change the answer. A deterministic router sends most issues to a cheap polish track and reserves an expensive two-sided trial for the issues that are genuinely contestable. The trial then returns one of three epistemic verdicts, and a recall safety net guards against the failure mode that a majority vote cannot self-detect.

## 5.1 Contestability routing

After the merge stage and ledger intake (status `raised`), every issue is routed by a deterministic function of its two derived labels (*kind*, *significance*), computed Node-side on the ledger row, not by an agent:

- `kind=mechanical` (any significance) → **polish**;
- `substantive` ∧ `minor` → **polish**;
- `substantive` ∧ `major` → **trial**.

A mechanical issue never reaches the trial regardless of its significance, and an unlabeled row defaults conservatively to `significance=major` / `kind=substantive`, so the failure direction is to over-adjudicate (waste a trial) rather than to under-scrutinise (skip one). Polish is an off-gate, never-drop track: mechanical issues get a batch copy-edit and minor-substantive ones a light-check, and an agent that is unavailable fails safe to `flagged` (queued) rather than dropping the issue. Contestability routing is the adjudication-side governor of contribution C1: only substantive-major issues incur the per-charge agent cost of the trial.

## 5.2 The two-sided five-tier trial

Each substantive-major charge is adjudicated by a trial with two decorrelated sides and a mostly deterministic verdict.

**Defense (whole-paper).** One defense agent per charge receives the *whole paper* plus the frozen spine and is instructed to scan the entire manuscript for prior treatment elsewhere. This provides the "addressed-elsewhere" catch at full strength: rather than a low-bar pre-screen, the defense reasons over the whole paper. Its grounds are drawn from a fixed enum {`addressed-in-text`, `out-of-scope`, `would-drift-anchor`, `severity-overstated`, `charge-stands`}.

**Jury (decorrelated, local-context).** A panel of *jurySize* jurors (default 5, code floor $\max(3, \textit{jurySize})$) votes {`valid`, `invalid`, `context-limited`}. Unlike the defense, each juror sees only *local* context—the claim spine plus the deterministic anchor-unit containing the charge's `evidence_anchor` plus any referenced unit—which is a quality choice (a focused reader is a sharper reader), not merely a cost one. Jurors are decorrelated by assigning each a distinct *framing* from a fixed list of 12; critically, the selection *always* includes the most-hostile adversarial framing (index 10) and the most-charitable peer framing (index 11) even at tier-5, where a naive prefix slice would omit both. A juror that votes `context-limited` with a stated `need` triggers on-demand context expansion (re-invoke with the requested unit, capped at *expansionsCap*, defaulting to the whole paper if the request cannot be targeted).

**Algorithm 1** Trial verdict rule (one charge; verbatim from `trial.workflow.js`)

*surviving* ← *tally.valid* + *tally.invalid* ▷ context-limited excluded
*quorum* ← $\lceil 0.8 \cdot \textit{jurySize} \rceil$
*decided* ← (*surviving* ≥ *quorum*)
*validFrac* ← *tally.valid*/*surviving*; *invalidFrac* ← *tally.invalid*/*surviving*
**if** *surviving* = 0 **then return** `author-required` ▷ all context-limited
**else if** *decided* ∧ *invalidFrac* > 0.6 **then return** `invalid-drop`
**else if** *decided* ∧ *validFrac* > 0.6 **then**
    *j* ← JudgeRoute(*charge*) ▷ routes only; does not re-judge
    **if** *j* unavailable **then return** `author-required`
    **end if**
    **if** *j.route* =`valid-fixable` ∧ *j.close_ criterion* non-empty **then return** `valid-fixable`
    **else return** `author-required` ▷ valid-fixable downgraded if no close criterion
    **end if**
**else if** *isFinalTier* **then return** `author-required` ▷ deadlock at tier-12
**else return** `escalate` ▷ re-run @ 12
**end if**

**Deterministic verdict.** Given the tally, the verdict is computed in code, not by a presiding agent. Only `valid`/`invalid` votes count toward *surviving* votes; `context-limited` votes are excluded so that "I could not tell" neither passes nor fails a charge. A judge agent is consulted only to *route* a charge the jury has already decided valid; it never re-litigates validity.

Two thresholds are deliberately distinct and must not be blurred: the *quorum* uses ≥ (*surviving* ≥ $\lceil 0.8 \cdot \textit{jurySize} \rceil$), while the *majority* is a strict > (*validFrac* > 0.6 or *invalidFrac* > 0.6 of surviving votes). When no clear majority forms at a non-final tier, the verdict is the transient directive `escalate`: the orchestrator re-batches those charges and re-runs the trial at *jurySize*=12 with *escalated*=`true`, at which point an undecided charge resolves to `author-required` (deadlock) rather than escalating again. `escalate` is only ever a verdict value, never a stored status. Both decorrelators (hostile and charitable framings) are present at every tier, including the 12-juror tier.

### 5.3 The three-way verdict and the "needs-you" asymmetry

The trial's terminal verdict space is three-way:

- `invalid-drop` — the charge is judged not real (hallucination, misreading, redundant, out-of-scope, or severity-inflated); it enters a non-terminal DROPS pool that recall re-checks before any drop becomes terminal.
- `valid-fixable` — valid, decidable from existing text, and safe; the judge attaches a `close_criterion` (one concrete sentence an edit must satisfy) and the drafter edits under the edit-safety guards (§6). A `valid-fixable` route is *code-enforced* to downgrade to `author-required` whenever the judge supplies no close criterion, so "fixable" is never unconditional.
- `author-required` — valid but needs author-private information, new data, or a research judgment call; it is routed to the human queue, accumulating across rounds to a single final (*zhongshen*) human pass. This is correct handling, not a recall loss.

The asymmetry that distinguishes PaperJury from forward-only tools lives in the two non-fix outcomes. A cooperative rewriter, biased toward emitting a changed output, structurally cannot return either `invalid-drop` ("this concern is not real") or `author-required` ("this needs you, and no

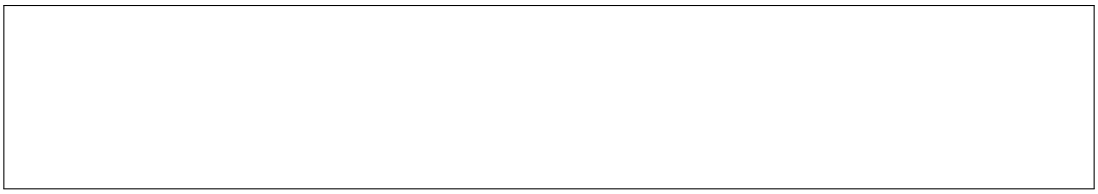

Figure 3: Adjudication flow: deterministic contestability routing sends mechanical and minor-substantive issues to the off-gate polish track and substantive-major issues to the five-tier trial; the trial's deterministic quorum/majority rule emits `invalid-drop` / `valid-fixable` / `author-required` (or the transient `escalate`), and the recall audit re-checks drops (Mode A) and strong-consensus majors (Mode B) before any edit is drafted.

edit can settle it") [19, 15]. By making "no fix / needs-you" first-class verdicts, the engine keeps research-level decisions with the author and declines to forcibly "fix" a critique that is either wrong or undecidable from the text alone.

### 5.4 Recall audit: a guard against correlated-wrong consensus

A quorum-plus-majority rule is only as good as the independence of its voters; a panel can agree and still be wrong together, and a majority vote cannot detect its own correlated error. The `recall-audit` stage runs after both trial and polish but before any edit, staffed by fresh skeptics decorrelated from the bodies whose output they check, in two modes:

- **Mode A (revive wrong drops).** Every drop is re-examined by *skeptics* parallel revive auditors with a *bias to revive*: any single skeptic reviving flips the drop. A trial-sourced drop revives to `escalate` (re-run @ 12), a polish-sourced drop to `re-trial` (@ 5); a drop no skeptic revives becomes a confirmed, terminal `dropped`.
- **Mode B (consensus spot-check).** The strong-consensus valid-fixable majors (those with $\textit{tally.valid} \geq 0.8 \cdot \textit{jury\_size}$ and *escalated*=`false`) are spot-checked with a *bias to flag*: any single skeptic finding the consensus correlated-wrong, or the planned fix harmful to the paper, routes the issue to `author-required` and removes it from the drafter's fixable set. A `hold` leaves it valid-fixable.

Mode B is the direct countermeasure to correlated-wrong consensus: precisely because a high-agreement majority is where an undetected shared error is most dangerous, those are the charges PaperJury re-examines with independent skeptics before committing an edit [4, 26]. Recall always runs as a fixed safety-envelope invariant.

## 6 Risk-Proportional Edit Safety and Author Sign-off

Once an issue is adjudicated `valid-fixable`, an LLM must change real LaTeX without drifting the paper's claims. Forward-only rewriters and self-refinement loops give no guarantee here: a model asked to "fix" a sentence can silently weaken a contribution or break a cross-table number, and self-correction without an external check is unreliable [15, 6]. PaperJury makes edit-safety *risk-proportional*: cheap deterministic guards run on every patch orchestrator-side, and an expensive semantic audit runs only on the patches those guards flag. The design of the spine, the anchor logic-transfer audit, and the minimal-edit policy is inspired by PaperSpine.

### 6.1 The spine: a frozen claim register

Before any edit, a one-time agent step extracts the paper's load-bearing sentences into a *spine* of up to seven anchor *types* treated as a checklist, not a quota: abstract-motivation, intro-problem, gap, contribution, methods-rationale, results-headline, and discussion-answer, forming one problem → solution → evidence → resolution arc. After author confirmation, `spine.js` freezes them deterministically (sequential `anchor_ids` `A1..An`, each resolved to a `passage_id` by first-match substring containment; missing types stay `not-yet-written`). **Anchors are never auto-edited.** The spine is the invariant against which every later edit is checked.

### 6.2 The deterministic LOW/RISKY pre-filter

Each drafted patch (*before*, *after*) is classified before it is applied by two dependency-free Node scripts. `anchor-diff.js` locates each frozen anchor in the current manuscript and flags it `need_audit` when the anchor is missing verbatim *or* its support region changed against a baseline ($need\_audit = \neg present \vee support\_changed$). The baseline is the frozen round-0 text, so the audit is *cumulative*: small per-round drifts that each pass individually are caught against round-0. `cross-ref.js` marks a patch RISKY iff a *changed salient token*—the symmetric difference of *salient*(*before*) and *salient*(*after*)—also appears (case-sensitive substring) in *another* passage. Salient tokens are decimals/percents/3+-digit numbers, `\ref`/`\cite`/`\label` keys, inline-math identifiers, CamelCase/ALLCAPS identifiers, and `\command` names minus a formatting denylist. Extraction is deliberately broad because a false RISKY costs only one cheap audit agent, whereas a false LOW is the dangerous direction. A LOW patch (no anchor flagged, no cross-ref hit) is applied directly; a flagged patch routes to a semantic audit.

### 6.3 Two semantic audits: meaning vs. edit

The audits are *separate* Workflows with distinct outputs and must not be conflated. **meaning-audit** runs only on flagged *frozen anchors*: per anchor a four-state verdict {`holds`, `weakened`, `contradicted`, `now-unsuppor` plus one arc-intactness pass over the full spine; `weakened` and `now-unsupported` are kept separate because they are different failures with different human calls. Any non-`holds` rolls back the *edit* that caused it (never the anchor) and queues it. **edit-audit** generalizes this to RISKY non-anchor edits with a two-state verdict {`holds`, `drift`} for in-place sense plus cross-section alignment; an unavailable audit agent defaults to `drift` (queue), the safe direction.

### 6.4 Apply, journal, revert, and the compile guard

`apply-patch.js` enforces an **exact-once** guard: *before* must occur exactly once (0 → `before-not-found`, >1 → `before-ambiguous`), rejecting blind edits. It writes the file and appends to an append-only `journal.jsonl`; `revert` applies the reverse patch and fails loudly if the after-text has moved, logging reverts as new `applied:false` entries rather than deletions. `compile-guard.js` is **detect-or-degrade**: with a LaTeX toolchain it really compiles and parses the log; without one it degrades to a structural lint and reports `compiled:null` (unknown, never a false "compiled"). In auto mode, `compiled:null` is not compile-passing: the patch is queued unless a real compile later verifies it. When a real compile detects breakage, the orchestrator performs the rollback via the journal. No issue is ever silently dropped: a terminal drop requires a recorded reason, and the polish/recall tracks re-check candidates before any drop becomes final.

### 6.5 Author sign-off and the auto-mode carve-out

The hard rule is that no manuscript edit is applied without explicit author sign-off. Review mode satisfies it per edit. Auto mode satisfies the *same* rule through up-front policy sign-off—spine confirmation, reviewer-assignment confirmation, and a pre-authorized bounded-aggressive apply policy—plus a return queue: a fix is auto-applied iff it addresses a major (or polish) item, satisfies its `close_criterion`, passes the edit-safety guard, stays within the per-passage rounds-touched cap (fixed at 2 across all intensities), and does not touch a spine anchor; otherwise it is queued. Nothing outside the authorized envelope is applied, and every queued item—anchor-touching, drift, compile-failed, or author-required—accumulates to a single human pass.

## 7 The Convergent Multi-Round Loop

Review (§3) runs one round and stops; the same engine becomes an unattended outer loop under Claude Code's `/goal`, iterating first-instance → $n$-th-instance → final-instance (*yishen* → *n-shen* → *zhongshen*) on the *current edited* paper until a deterministic convergence. Two design choices make the loop both honest and terminating: each round is *clean* (maximally decorrelated from the ledger), and convergence is decided by a deterministic ledger query rather than a model's self-assessment of being done [6, 20].

### 7.1 Clean rounds: re-review as the fix test

The reviewer-facing steps—`assign-reviewers`, `reading-check`, `coverage-auditor`—never see the cumulative ledger or any prior open question; only the deterministic spine carries forward. This buys two properties for free. First, an independent re-raise of an issue in a later round is *corroboration*, not anchoring: a fresh panel that never saw the first complaint and files it again is genuine agreement, whereas a panel primed with the ledger would merely echo it [4, 16]. Second, a clean re-review of an edited passage *is* the operational "did the edit actually fix it" test—the new round either re-raises the issue (the edit failed) or does not (the edit held), with no agent asked to grade its own prior fix. The inner reading loop's notion of "dry" (no new *issue*) and the outer loop's "dry" (no new *applied edit*) are nested and distinct; we never conflate them.

### 7.2 The durable ledger and the deterministic clerk merge

A single cumulative `LEDGER.json` is the cross-round source of truth; each clean round produces a fresh row set that the round-boundary `clerk` (*shujiguan*) reconciles into it. The clerk runs two jobs. RECONCILE judges each carried open question (a prior-round `author-required`, `queued`, or `valid-fixable` row) against this round's applied edits and the current paper, returning `closed`, `invalidated`, or `still-open`; an agent that does not respond defaults to `still-open` (safe). DEDUP matches each new issue against the carried rows.

The merge *key* is deterministic, which is what prevents a semantic clerk from becoming a hidden second source of truth. A new row folds into a carried row only when it shares the same `passage_id` *and* the clerk's same-issue confidence is at least *simThreshold*=0.7:

$$\mathsf{merge} \iff \big(\mathsf{passage\_id}_{\mathrm{new}} = \mathsf{passage\_id}_{\mathrm{cand}}\big) \ \wedge \ \big(\mathsf{confidence} \geq 0.8\big).$$

The agent judges only similarity; the gate is code. Borderline cases bias to *genuinely-new* (a missed merge re-adds a recoverable row; a wrong merge hides a distinct issue), so convergence stays an honest "nothing new" test rather than a proxy. A folded re-raise becomes status `withdrawn`

(noted `merged into` ⟨*id*⟩), never silently dropped, and bumps the target's `raised_by_count`—again recording corroboration.

### 7.3 The completion fact: a ledger query, not a judgment

Completion is a deterministic fact. The `/goal` gate (`node ledger.js gate`) is evaluated over the same ledger state that the engine's own adjudication and apply steps write, and passes iff $\mathsf{gate\_blocking\_major} = 0$ over the gate-blocking active states {`raised`, `in-trial`, `re-trial`, `valid-fixable`}; `author-required` is deliberately *excluded*, so the loop can wind down with real questions still queued for a single *zhongshen* (final) human pass. To stop budget exhaustion or a stalled trial from *faking* completion, an additional check requires `node ledger.js unadjudicated` to be empty—no active major in {`raised`, `in-trial`, `re-trial`} may carry a null verdict. A round is complete only when both hold; an independent Haiku evaluator under `/goal` checks these deterministic ledger facts and never re-runs the semantic audits, so the verdict is structural rather than a model's opinion of its own work [26, 6].

### 7.4 Convergence and hard limits

The loop terminates on the *first* of three conditions:

1. **Clerk convergence** (primary): the clerk reports $\mathsf{genuinely\_new\_count} = 0$, $\mathsf{new\_closures\_count} = 0$, and $\mathsf{new\_author\_required\_count} = 0$—a clean round that changed nothing.
2. **Applied-quiescence** (backstop): $K$ consecutive rounds with zero *applied* edits ($K = \mathit{dryStop}$); any applied edit resets the counter, and nits/queued items—which never journal as applied—do not.
3. **Hard limits**: `max_rounds`, optional wall-clock, or an AFK ceiling of $H$ hours.

Termination keys on the applied-edit channel and convergence, never on queue-quiescence: an adversarial fresh panel can always queue one more concern, so the queue may keep growing while the paper is in fact stable. On a hard stop with majors still active, they are flipped to `queued` so the gate winds down clean. `author-required` rows accumulate across all rounds with no per-round human interrupt and are handed off in one *zhongshen* (final) pass, where the human signs off survivors (the wind-down never auto-applies, and a queued item is never silently dropped—`ledger.js set ... dropped` enforces a reason). The result is a bounded, unattended loop whose stopping point is a verifiable property of the ledger, not a claim the system makes about itself.

## 8 Implementation on an Agent Harness

PaperJury is implemented as a single Claude Code skill. This section describes how the paradigm of Section 3 maps onto one concrete agent harness, and which robustness rails turn an unreliable batch-of-agents substrate into a system that fails safe.

### 8.1 Skill + Workflow + Memory

The three paradigm primitives bind to three harness mechanisms. The *Skill* is the loaded entry point and methodology. *Workflows* are the fan-out engine: each semantic, no-human-in-the-middle step (`assign-reviewers`, `reading-check`, `coverage-auditor`, `merge`, `trial`, `polish`, `recall-audit`, `drafter`, `edit-audit`, `meaning-audit`, `clerk`) is a Workflow whose sandboxed agents run in parallel with schema-validated output by construction. *Memory* holds durable state: the ledger

(`LEDGER.json` machine truth, rendered `LEDGER.md` view) plus Claude project memory for house style, venue, and persona tuning across sessions.

### 8.2 The deterministic/sandbox boundary

The Workflow sandbox has *no filesystem and no subprocess*. This single constraint forces the architecture: any guard that must read or write a file, or spawn a compiler, cannot live in a Workflow. All trustworthy guards therefore run *orchestrator-side* as Node scripts invoked via Bash between Workflow calls (`decompose`, `ledger`, `journal`, `apply-patch`, `anchor-diff`, `cross-ref`, `spine`, `compile-guard`, `compliance-check`). The liability becomes the load-bearing isolation property of contribution C2: the layer that decides whether an edit is applied, and whether a round is complete, is deterministic code an agent cannot reason past. Semantic Workflows receive all inputs *inline* in `args`; because this harness delivers `args` as a JSON *string*, every Workflow parses defensively (`typeof args === 'string' ? JSON.parse(args) : args`). The data contract is the set of 14 named orchestrator enrichment seams, and at apply time the *ledger row is the single source of truth*: the drafter emits neither `passage_id`, `close_criterion`, nor `round`, which the orchestrator supplies from the row (SEAM 4).

### 8.3 Schema-validated structured outputs

Every Workflow agent returns via the harness's structured-output facility, so each emission is validated against a declared schema before the orchestrator reads it, letting downstream Node code treat agent output as data rather than prose to re-parse. It is also the source of one failure mode the rails must absorb: under a transient rate-limit a schema-forced agent can *complete without emitting* structured output, so an entire batch returns empty. The system never trusts an agent to echo identifiers it could fabricate: `reviewer_id` (R1…RN), `charge_id`, anchor ids, and all merge aggregates are derived in deterministic code, and coverage-auditor flags are validated against the real `(reviewer_id, section)` set rather than taken at the agent's word.

### 8.4 Robustness rails

A batch of sandboxed agents is unreliable at volume, so the orchestrator wraps every fan-out in deterministic rails (Table 1). **Idempotent retry** is keyed on state: per-charge retry asks the ledger which `charge_id` still lacks a verdict, and charge-less phases (clerk, coverage-auditor, merge) retry by `(phase, round)`; a re-run never re-judges an already-adjudicated charge. A **canary** batch of 3–5 precedes any large run, because a single-agent probe does not predict a batch (the limit is sustained concurrent volume). A **Monitor watchdog** kills, backs off, and re-runs a sub-batch after ∼90 s of no progress. **Phase-bounded batching** keeps each invocation inside one phase of the sequential DAG; when *charges* × *agents-per-charge* would exceed MAX_AGENTS_PER_WF ≈ 600, the trial phase splits into same-phase invocations of $\lfloor 600/(\mathit{jury\_size}+2) \rfloor$ charges each (one defense and one judge accompany each jury). The **rate-limit policy** is canary-heartbeat pause/resume; $K$ consecutive failed canaries trigger queue-notify and pause, and a hard ceiling of $H$ hours winds the run down to the queue. Concurrency is capped at 16 simultaneous agents per invocation, with a 1000-agent lifetime cap *per invocation*; `/goal` hosts a sequence of invocations and never runs concurrent top-level Workflows.

| Rail | Mechanism |
|---|---|
| Idempotent retry | re-run keyed on unverified `charge_id` / `(phase,round)` |
| Canary | probe batch of 3–5 before any large run |
| Monitor watchdog | ∼90 s no-progress → kill, back off, re-run sub-batch |
| Phase-bounded | one DAG phase per invocation; never splice phase tails |
| Batch split | $\lfloor 600/(jury\_size{+}2) \rfloor$ charges/invocation |
| Rate-limit | canary-heartbeat pause/resume; $K$ fails → pause; $H$ h → queue |
| Concurrency | 16 agents/invocation; 1000 lifetime cap/invocation |

Table 1: Deterministic robustness rails wrapping the semantic fan-out. Exact thresholds are reproduced from the implementation; $K$ and $H$ are policy parameters set at run start.

### 8.5 Model and effort policy

All engine agents run on a single frontier model (Opus 4.8, inherited or `model:'opus'`); no Haiku is used for engine agents. The lone exception is the `/goal` completion evaluator, which checks deterministic ledger facts only and never re-runs a semantic audit, so a cheaper model is appropriate there. Reasoning effort is set at session level (maximal where available, otherwise xhigh). This concentrates compute on accuracy-bearing layers, consistent with the trilemma's corollary that precision comes from the verify layer, not from agent count (contribution C1).

### 8.6 Generic by construction

The skill carries *zero hardcoded paths and no bundled project files*: no venue list, no deterministic venue detector, and no template (an agent infers the venue family from `\documentclass` and asks when unclear; the official template source is reported as a link for human confirmation, never swapped in). Every input is resolved at runtime: the manuscript directory, the default ledger directory `<manuscript-dir>/.paper-review/`, the canonical section list and cross-round-stable `passage_id`s from `decompose`, and the project-owned `constraints.json` consumed by Part A desk-reject screening. `compile-guard` follows the same *detect-or-degrade* contract: a real LaTeX compile when a toolchain is on `PATH`, otherwise `compiled: null` (unknown), never a false success. The result is portable across CS venues and machines without bundled project files or hardcoded project paths; project-owned constraints remain explicit runtime inputs.

## 9 Evaluation Methodology

This section states the dimensions and metrics by which a system of PaperJury's shape is assessed. The evaluation framework follows established LLM-as-a-judge and AI-feedback methodology [13, 5, 11, 12], and is organized as a two-arm expert-review design with five measurement dimensions tied to the five contributions.

### 9.1 Two-arm expert-review design

The assessment uses two complementary arms. In **Arm 1 (issue quality)**, domain experts review a held-out set of CS papers to produce a human issue panel per paper; PaperJury reviews the same papers; the two issue sets are compared. In **Arm 2 (system review)**, the same experts review PaperJury *itself*—inspecting its verdicts, edits, and drops—to assess whether its routing decisions are ones a human reviewer would endorse, addressing the known risk that a model judge favors its own framing [16, 26].

| Dimension | Reference | Tests |
|---|---|---|
| Issue quality | Expert panel (Arm 1) | C1 |
| Verdict accuracy | Expert labels | C1, C3 |
| Edit-safety violation | Expert audit (Arm 2) | C4 |
| Convergence behavior | Ledger / clerk facts | C5 |
| Cost / attention | Unbounded generator | C1 |

Table 2: Evaluation framework. Each dimension is defined as a measurement against a human or deterministic reference.

### 9.2 Measurement dimensions

- **Issue quality vs. expert judgment.** For each paper, match PaperJury's raised issues against the human panel (by passage and semantic equivalence) and report precision (raised issues an expert endorses) and recall (expert issues PaperJury catches), separated by the three-way verdict so that an accurate `invalid-drop` counts as a success rather than a false positive (C1).
- **Verdict / routing accuracy.** On the subset that reaches the 5-tier trial, measure agreement between the engine's verdict (`invalid-drop` / `valid-fixable` / `author-required`) and the experts' label for the same charge. This is the central test of the contestability-routing claim (C1, C3).
- **Edit-safety violation rate.** Over applied `valid-fixable` edits, measure the rate at which an edit drifts a claim—an expert detects meaning change in an anchor or a cross-section inconsistency that the anchor-bounded guard (meaning-audit / edit-audit, compile-guard) failed to catch. The guard's revert-and-queue policy is designed to drive this rate toward zero (C4).
- **Convergence behavior.** Across the outer loop, report rounds to termination, which predicate fired first (clerk convergence vs applied-quiescence vs hard limit), genuinely-new and new-closure counts per round, and whether a clean re-review confirms that an applied edit fixed its issue (C5).
- **Cost / attention.** Report agents spawned, tokens, and wall-clock per paper against the naive unbounded per-(unit $\times$ lens) generator, to test the trilemma claim that the binding constraint is cost rather than the precision/recall tradeoff (C1).

Table 2 maps each measurement dimension to its reference and the contributions it tests.

**Threats to validity.** **Construct:** a silver ground truth from author self-revision under-counts issues the author also missed, so it bounds recall from below; an external expert panel is needed for an unbiased reference. **Internal:** model judges can recognize and favor their own generations [16] and exhibit sycophancy [19], so Arm 2 reviewers must be blind to which verdict the engine assigned, and the recall safety net (Mode A revive, Mode B correlated-wrong spot-check) must itself be audited for correlated error. **External:** a single paper or single venue family does not generalize; the study must span the Vision, NLP, and ML families. **Reliability:** because rate-limiting can wipe a batch, a timed result must report the canary, retry, and watchdog policy under which it was obtained, lest a transient failure be read as a capability ceiling. Finally, because no edit is applied outside the up-front-authorized envelope and anchors are never auto-edited, the study measures a system

whose safety behaviors are *enforced policies* grounded in code and protocol; whether those policies yield safe *outcomes* in practice is what the edit-safety violation rate is meant to establish.

# 10 Discussion, Limitations, and Responsible Use

## 10.1 Limitations

**Single model family.** All engine agents run on one model family (Opus 4.8), with a single Haiku exception for the `/goal` completion evaluator, which checks only deterministic ledger facts. We have not characterized how the panel, jury, and audits behave under other models, nor whether decorrelation across framings substitutes for decorrelation across architectures. Self-preference among same-family evaluators [16] is a known risk we mitigate structurally (isolation, a two-sided trial, fresh decorrelated skeptics) but have not measured.

**Cost and attention.** Although this paper (§1) treats cost/attention as the binding constraint, the engine is still agent-heavy: a trial phase batches up to $\approx 600$ agents per Workflow invocation under a 1000-agent lifetime cap and 16-way concurrency. A large fan-out also remains exposed to rate-limiting, which can wipe a batch; the generation- and adjudication-side governors are designed to keep a real full-paper run within budget, and the robustness rails (Table 1) are designed to fail safe under transient limits.

**Persona realism.** Reviewer quality depends on personas we do not validate against human reviewers. The shipped degrade path emits a generic "general CS (gatekeeper)" persona rather than the named domain triad of our design notes, and model critics are known to diverge from human judgment [11, 9]. We make no claim that our jurors approximate a real program committee.

## 10.2 Responsible use

PaperJury is a pre-submission self-check, *not* a peer-review replacement, and we state this as policy. By construction it does not invent missing experiments, fabricate results, add unsupported claims, or hide limitations: issues that need new data, private knowledge, or research judgment route to *author-required* and accumulate to a single human pass, never silently fixed nor dropped (a drop requires a logged reason). Research decisions stay with the author. Autonomy is bounded by enforced policy, not by trust: auto mode is entered explicitly and never self-detected; no edit is applied outside an up-front-authorized envelope; spine anchors are never auto-edited; and the completion fact is a deterministic ledger query, not a model self-judgment that could rationalize a sycophantic "done" [19, 6]. These guardrails reduce, but do not eliminate, the risk of an over-confident or wrong edit; a human signs off survivors at the *zhongshen* (final) pass.

## 10.3 Broader applicability

The paradigm's load-bearing pieces are domain-agnostic: a deterministic/semantic split forced by an isolated execution sandbox, contestability routing that spends deliberation only where it changes the answer, a two-sided adjudication with a verdict that can return "no fix," and risk-proportional anchor-bounded edits over a frozen claim spine. Any artifact with checkable structure and contestable claims—grant proposals, specifications, legal or policy documents, code review—fits the same shape: fan out holistic critics, route by contestability, adjudicate with due process, apply

only bounded audited changes. We leave such transfers to future work and claim no demonstrated generality.

## 11 Conclusion

PaperJury is a Claude Code skill that brings adversarial, due-process pre-submission review and bounded auto-revision to LaTeX CS papers. Its design rests on a single closed-loop idea: rather than push a manuscript forward, route *every* issue through contestability-gated routing, a quorum-decided two-sided trial with a three-way verdict (`valid-fixable` / `author-required` / `invalid-drop`), anchor-bounded edit-safety, and a deterministically-converged unattended outer loop. The terminal verdict space includes `invalid-drop`, the "no fix" move a forward-only rewriter cannot make and a reviewer can. Architecturally, the binding constraint is reframed from precision/recall to COST/attention, governed at generation and adjudication; all trustworthy guards run orchestrator-side in Node between schema-validated semantic fan-out, joined by named enrichment seams with the ledger row as the single source of truth at apply time.

More broadly, the paradigm's load-bearing pieces are domain-agnostic: a deterministic/semantic split forced by an isolated execution sandbox, contestability routing that spends deliberation only where it changes the answer, a two-sided adjudication with a verdict that can return "no fix," and risk-proportional anchor-bounded edits over a frozen claim spine. These pieces suggest a reusable pattern beyond the LaTeX CS paper that motivates it here, but the present paper claims only that design hypothesis rather than demonstrated cross-domain generality.

## References


[1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback, 2022. URL https://arxiv.org/abs/2212.08073. arXiv preprint arXiv:2212.08073.

[2] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.07201.

[3] Mike D'Arcy, Tom Hope, Larry Birnbaum, and Doug Downey. Marg: Multi-agent review generation for scientific papers, 2024. URL https://arxiv.org/abs/2401.04259. arXiv preprint arXiv:2401.04259.

[4] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2305.14325.

[5] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Yuanzhuo Wang, and Jian Guo. A survey on llm-as-a-judge, 2024. URL https://arxiv.org/abs/2411.15594. arXiv preprint arXiv:2411.15594.

[6] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *Interna-*

*tional Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.01798.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. URL https://arxiv.org/abs/2202.03629.

[8] Yiqiao Jin, Qinlin Zhao, Yiyang Wang, Hao Chen, Kaijie Zhu, Yijia Xiao, and Jindong Wang. Agentreview: Exploring peer review dynamics with llm agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://arxiv.org/abs/2406.12708.

[9] Dongyeop Kang, Waleed Ammar, Bhavana Dalvi, Madeleine van Zuylen, Sebastian Kohlmeier, Eduard Hovy, and Roy Schwartz. A dataset of peer reviews (peerread): Collection, insights and nlp applications. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2018. URL https://arxiv.org/abs/1804.09635.

[10] Mina Lee, Percy Liang, and Qian Yang. Coauthor: Designing a human-ai collaborative writing dataset for exploring language model capabilities. In *CHI Conference on Human Factors in Computing Systems*, 2022. URL https://arxiv.org/abs/2201.06796.

[11] Weixin Liang, Yuhui Zhang, Hancheng Cao, Binglu Wang, Daisy Ding, Xinyu Yang, Kailas Vodrahalli, Siyu He, Daniel Scott Smith, Yian Yin, Daniel A. McFarland, and James Zou. Can large language models provide useful feedback on research papers? a large-scale empirical analysis, 2023. URL https://arxiv.org/abs/2310.01783. arXiv preprint arXiv:2310.01783; also NEJM AI 2024.

[12] Ryan Liu and Nihar B. Shah. Reviewergpt? an exploratory study on using large language models for paper reviewing, 2023. URL https://arxiv.org/abs/2306.00622. arXiv preprint arXiv:2306.00622.

[13] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2303.16634.

[14] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. URL https://arxiv.org/abs/2408.06292. arXiv preprint arXiv:2408.06292.

[15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2303.17651.

[16] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. Llm evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2404.13076.

[17] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2302.04761.

[18] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025. URL https://arxiv.org/abs/2501.04227.

[19] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, Shauna Kravec, Timothy Maxwell, Sam McCandlish, Kamal Ndousse, Oliver Rausch, Nicholas Schiefer, Da Yan, Miranda Zhang, and Ethan Perez. Towards understanding sycophancy in language models, 2023. URL https://arxiv.org/abs/2310.13548. arXiv preprint arXiv:2310.13548.

[20] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2303.11366.

[21] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291. arXiv preprint arXiv:2305.16291.

[22] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024. URL https://arxiv.org/abs/2308.11432.

[23] Yixuan Weng, Minjun Zhu, Guangsheng Bao, Hongbo Zhang, Jindong Wang, Yue Zhang, and Linyi Yang. Cycleresearcher: Improving automated research via automated review. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2411.00816.

[24] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob N. Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL https://arxiv.org/abs/2504.08066. arXiv preprint arXiv:2504.08066.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

[26] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685.